

Fundamentals of Data Science

Question 1: Student Subject Averages using NumPy

```
2] |
Os  ▶ import numpy as np
    student_scores = np.array([[80, 75, 85, 70],
                               [90, 85, 80, 75],
                               [70, 80, 90, 85],
                               [85, 90, 95, 80]])

    subjects = ["Math", "Science", "English", "History"]
    average = np.mean(student_scores, axis=0)
    for i in range(4):
        print(subjects[i], "Average =", average[i])
    highest = np.argmax(average)
    print("Highest average subject:", subjects[highest])

... Math Average = 81.25
   Science Average = 82.5
   English Average = 87.5
   History Average = 77.5
   Highest average subject: English
```

Question 2: Average Product Price using NumPy

```
3] |
Os  ▶ import numpy as np
    prices = np.array([[100, 120, 150],
                       [80, 90, 110],
                       [200, 180, 160]])

    average_price = np.mean(prices)
    print("Average price =", average_price)

... Average price = 132.22222222222223
```

Question 3: Average Sale Price of Houses with More Than 4 Bedrooms

```
5] |
s  ▶ import numpy as np
    # Columns: Bedrooms, Area, Sale Price
    house_data = np.array([[3, 1200, 250000],
                           [5, 1800, 400000],
                           [6, 2200, 500000],
                           [4, 1500, 320000],
                           [5, 2000, 450000]])

    selected = house_data[house_data[:, 0] > 4]
    average_price = np.mean(selected[:, 2])
    print("Average sale price =", average_price)

... Average sale price = 450000.0
```

Question 4: Total Yearly Sales and Percentage Increase

```
[5]
✓ Os
import numpy as np
sales_data = np.array([10000, 12000, 15000, 18000])
total_sales = np.sum(sales_data)
percentage_increase = ((sales_data[3] - sales_data[0]) / sales_data[0]) * 100
print("Total sales =", total_sales)
print("Percentage increase =", percentage_increase, "%")

... Total sales = 55000
Percentage increase = 80.0 %
```

Question 5: Average Fuel Efficiency and Percentage Improvement

```
[6]
✓ Os
import numpy as np
fuel_efficiency = np.array([25, 30, 28, 35, 32])
average = np.mean(fuel_efficiency)
old_model = fuel_efficiency[0]
new_model = fuel_efficiency[1]
improvement = ((new_model - old_model) / old_model) * 100
print("Average fuel efficiency =", average)
print("Percentage improvement =", improvement, "%")

... Average fuel efficiency = 30.0
Percentage improvement = 20.0 %
```

Question 6: Total Grocery Cost with Discount and Tax

```
[7]
✓ Os
prices = [100, 50, 200]
quantities = [2, 3, 1]
discount_rate = 10
tax_rate = 5
subtotal = 0
for price, quantity in zip(prices, quantities):
    subtotal += price * quantity
discount = subtotal * discount_rate / 100
after_discount = subtotal - discount
tax = after_discount * tax_rate / 100
total = after_discount + tax
print("Subtotal =", subtotal)
print("Discount =", discount)
print("Tax =", tax)
print("Total cost =", total)

... Subtotal = 550
Discount = 55.0
Tax = 24.75
Total cost = 519.75
```

Question 7: Pandas Order Analysis

```
[8]
✓ 1s
import pandas as pd

order_data = pd.DataFrame({
    "Customer_ID": [1, 1, 2, 2, 3],
    "Order_Date": [
        "2026-01-02",
        "2026-01-05",
        "2026-01-03",
        "2026-01-10",
        "2026-01-08"
    ],
    "Product": ["Pen", "Book", "Pen", "Bag", "Book"],
    "Quantity": [2, 3, 5, 1, 4]
})

print("Orders by customer:")
print(order_data.groupby("Customer_ID").size())

print("\nAverage quantity by product:")
print(order_data.groupby("Product")["Quantity"].mean())

order_data["Order_Date"] = pd.to_datetime(order_data["Order_Date"])

print("\nEarliest date:", order_data["Order_Date"].min())
print("Latest date:", order_data["Order_Date"].max())
```

... Orders by customer:

Customer_ID	
1	2
2	2
3	1

dtype: int64

Average quantity by product:

Product	
Bag	1.0
Book	3.5
Pen	3.5

Name: Quantity, dtype: float64

Earliest date: 2026-01-02 00:00:00
Latest date: 2026-01-10 00:00:00

Question 8: Top 5 Products Sold

```
[9]
✓ 0s
import pandas as pd

sales = pd.DataFrame({
    "Product": ["Pen", "Book", "Pen", "Bag", "Book", "Pen", "Bag"],
    "Quantity": [10, 8, 15, 5, 12, 20, 9]
})

top5 = sales.groupby("Product")["Quantity"].sum()
top5 = top5.sort_values(ascending=False).head(5)

print(top5)
```

... Product

Pen	45
Book	20
Bag	14

Name: Quantity, dtype: int64

Question 9: Property Data Analysis using Pandas

```
[10]
✓ Os  import pandas as pd

property_data = pd.DataFrame({
    "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],
    "Bedrooms": [3, 5, 4, 6, 5],
    "Area": [1200, 1800, 1500, 2200, 2000],
    "Price": [300000, 450000, 350000, 500000, 480000]
})

print("Average price by location:")
print(property_data.groupby("Location")["Price"].mean())

print("\nProperties with more than 4 bedrooms:")
print((property_data["Bedrooms"] > 4).sum())

largest = property_data.loc[property_data["Area"].idxmax()]

print("\nProperty with largest area:")
print(largest)
```

... Average price by location:

Location	Price
Chennai	410000.0
Madurai	425000.0

Name: Price, dtype: float64

Properties with more than 4 bedrooms:

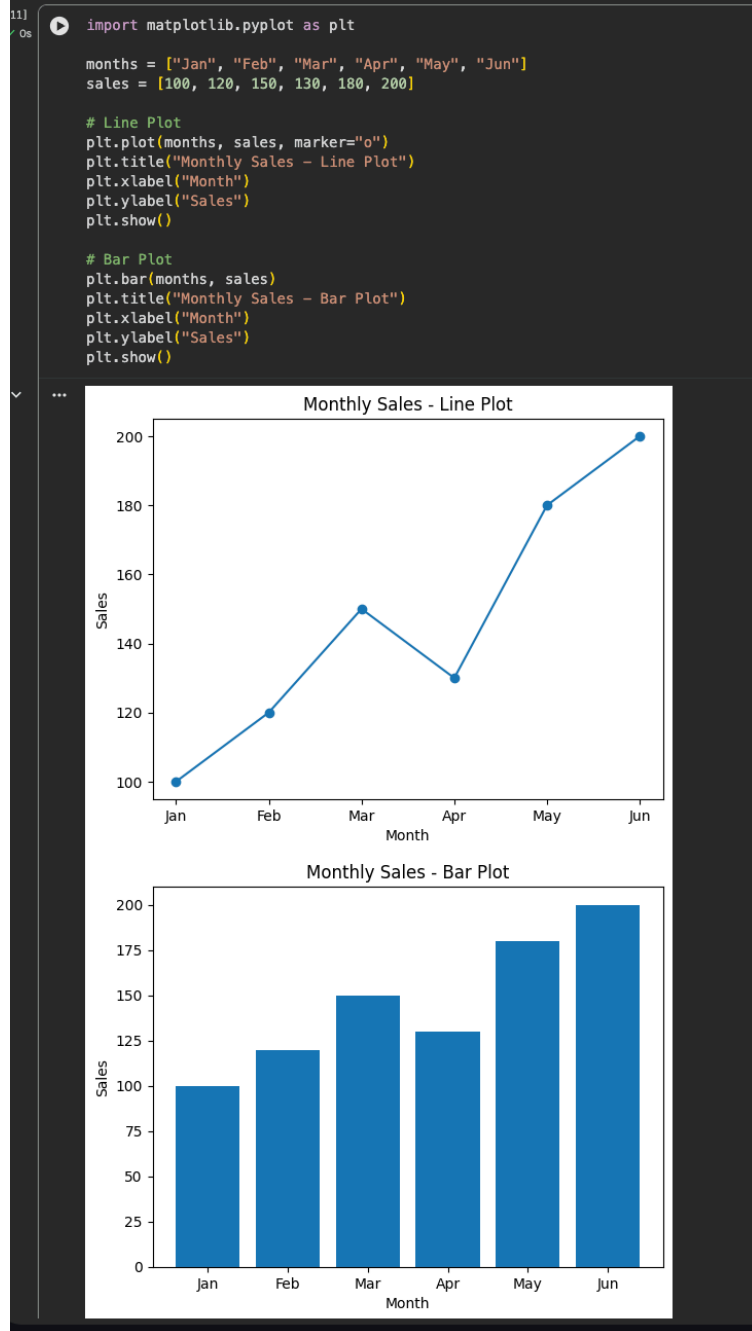
3

Property with largest area:

Location	Bedrooms	Area	Price
Madurai	6	2200	500000

Name: 3, dtype: object

Question 10: Monthly Sales – Line Plot and Bar Plot



Question 11: Sales over Time – Line, Scatter and Bar Plots

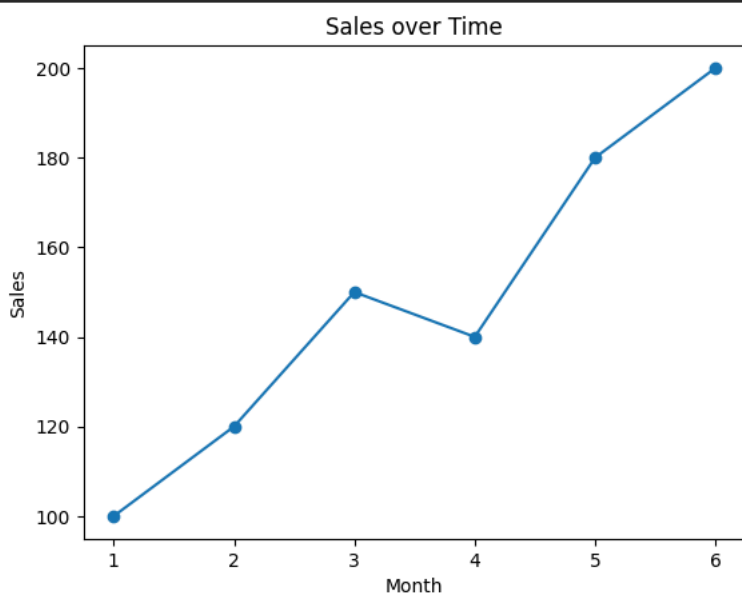
```
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5, 6]
sales = [100, 120, 150, 140, 180, 200]

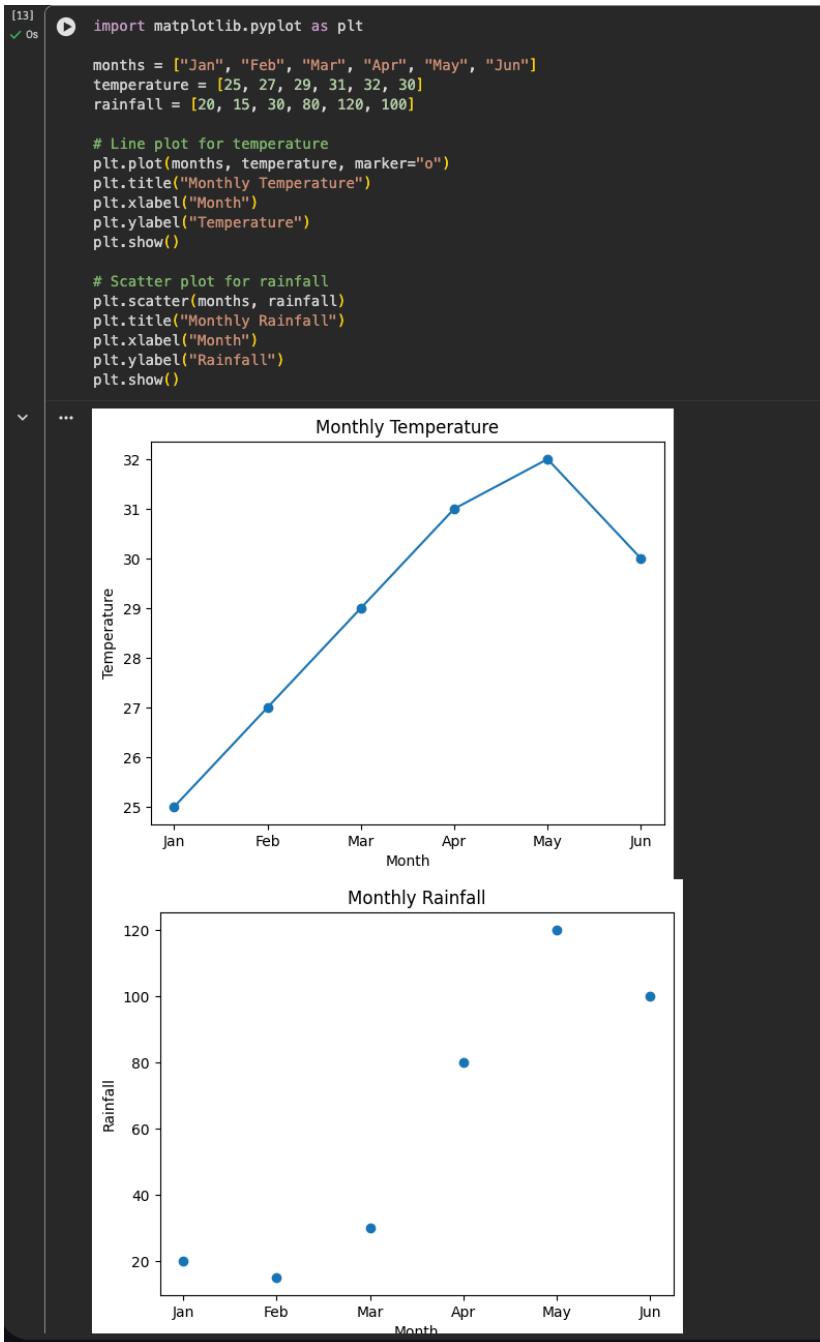
# Line Plot
plt.plot(months, sales, marker="o")
plt.title("Sales over Time")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

# Scatter Plot
plt.scatter(months, sales)
plt.title("Sales - Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

# Bar Plot
plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



Question 12: Monthly Temperature and Rainfall Plots



Question 13: Word Frequency from a Text File

```
DS1.py (3.14.5)
from collections import Counter
import re

with open("DS1.txt", "r") as file:
    text = file.read().lower()

words = re.findall(r"\b\w+\b", text)

frequency = Counter(words)

for word, count in frequency.most_common():
    print(word, ":", count)
```

```
IDLE Shell 3.14.5
Python 3.14.5 (v3.14.5:5607950ef23, May 10 2026, 07:38:09) [Clang 21.0.0 (clang-2100.0.123.102)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/devishree/Desktop/DS1.py =====
python : 2
is : 2
easy : 1
powerful : 1
>>>
```

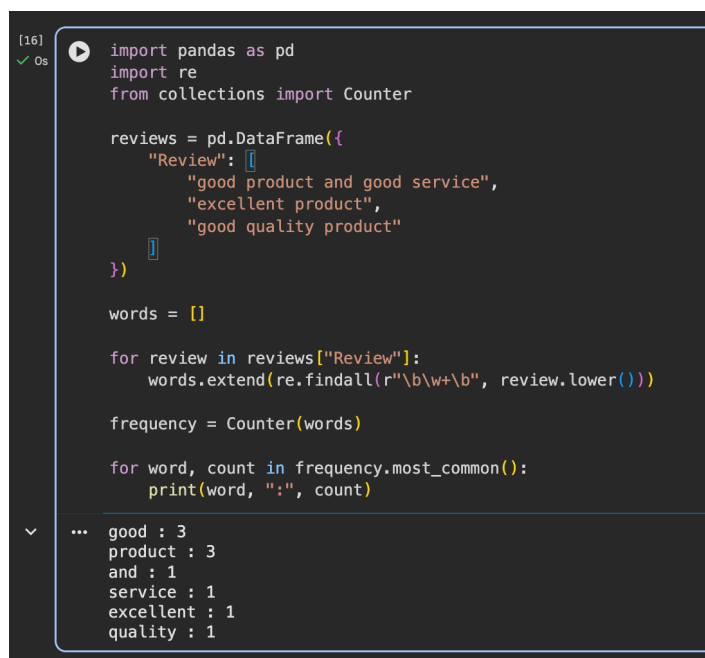
Question 14: Frequency Distribution of Customer Ages



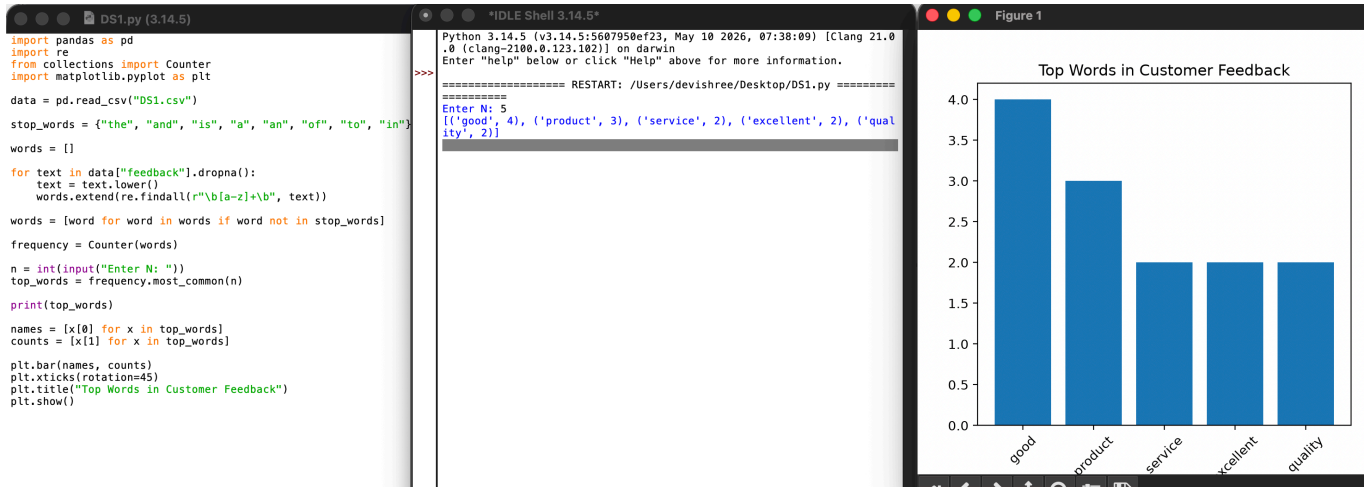
Question 15: Frequency Distribution of Likes



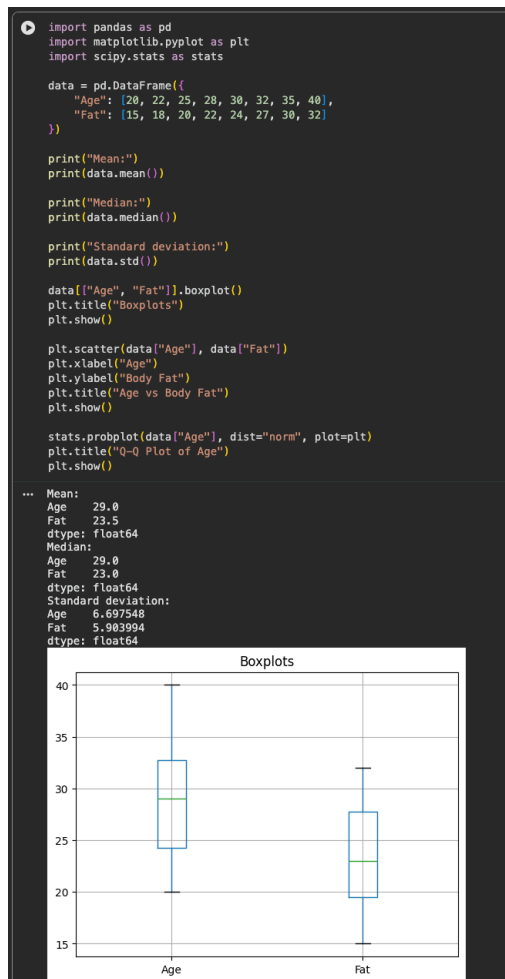
Question 16: Word Frequency in Customer Reviews



Question 17: Customer Feedback – Top N Word Frequency



Question 18: Mean, Median, Standard Deviation, Boxplot, Scatter Plot and Q-Q Plot



Question 19: 95% Confidence Interval for Blood Pressure Reduction

```
[18] import numpy as np
      from scipy import stats

      drug = np.array([8, 10, 12, 9, 11, 13, 10, 12])
      placebo = np.array([3, 5, 4, 6, 2, 5, 4, 3])

      def confidence_interval(data):
          mean = np.mean(data)
          sem = stats.sem(data)

          interval = stats.t.interval(
              0.95,
              len(data) - 1,
              loc=mean,
              scale=sem
          )

          return interval

      print("Drug 95% CI:", confidence_interval(drug))
      print("Placebo 95% CI:", confidence_interval(placebo))

... Drug 95% CI: (np.float64(9.216289685290807), np.float64(12.033710314709193))
Placebo 95% CI: (np.float64(2.9053916697328868), np.float64(5.094608330267113))
```

Question 20: A/B Test – Compare Two Conversion Rate Samples

```
[19] import numpy as np
      from scipy.stats import ttest_ind

      design_A = np.array([0.10, 0.12, 0.11, 0.13, 0.09, 0.12])
      design_B = np.array([0.14, 0.15, 0.13, 0.16, 0.12, 0.14])

      t_stat, p_value = ttest_ind(design_A, design_B)

      print("t-statistic =", t_stat)
      print("p-value =", p_value)

      if p_value < 0.05:
          print("Statistically significant difference")
      else:
          print("No statistically significant difference")

... t-statistic = -3.3999999999999995
p-value = 0.006771072424337956
Statistically significant difference
```

Question 21: Point Estimate and Confidence Interval from CSV

```
DS1.py (3.14.5)
import pandas as pd
import numpy as np
from scipy import stats

data = pd.read_csv("DS1.csv")
values = data.iloc[:, 0].dropna()

n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level (for example 0.95): "))
precision = float(input("Enter desired precision: "))

sample = values.sample(n=min(n, len(values)), random_state=1)
mean = sample.mean()
sem = stats.sem(sample)

ci = stats.t.interval(
    confidence,
    len(sample) - 1,
    loc=mean,
    scale=sem
)

print("Point estimate =", mean)
print("Confidence interval =", ci)
print("Desired precision =", precision)
```

```
IDLE Shell 3.14.5
Python 3.14.5 (v3.14.5:5607950ef23, May 10 2026, 07:38:09) [Clang 21.0.0 (clang-2100.0.123.102)] on darwin
Enter "help" below or click "Help" above for more information.

===== RESTART: /Users/devishree/Desktop/DS1.py =====
Enter sample size: 5
Enter confidence level (for example 0.95): 0.95
Enter desired precision: 1
Point estimate = 12.44
Confidence interval = (np.float64(11.718124272648573), np.float64(13.161875727351426))
Desired precision = 1.0
```

Question 22: Confidence Interval for Mean Customer Rating

```
DS1.py (3.14.5)
import pandas as pd
from scipy import stats

data = pd.read_csv("DS1.csv")
ratings = data["rating"].dropna()

confidence = 0.95
mean = ratings.mean()
sem = stats.sem(ratings)

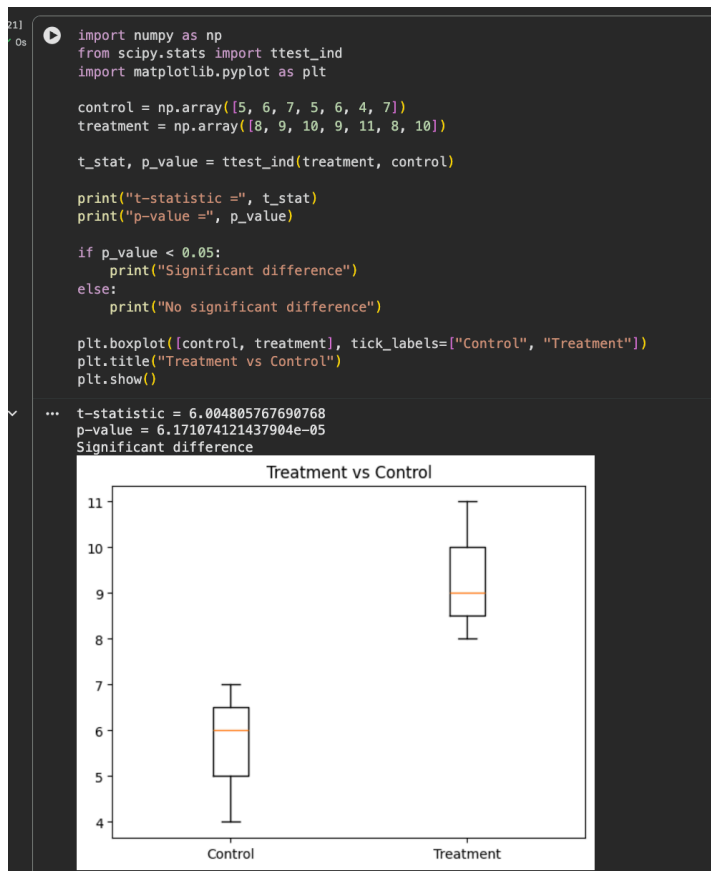
ci = stats.t.interval(
    confidence,
    len(ratings) - 1,
    loc=mean,
    scale=sem
)

print("Mean rating =", mean)
print("95% Confidence interval =", ci)
```

```
IDLE Shell 3.14.5
Python 3.14.5 (v3.14.5:5607950ef23, May 10 2026, 07:38:09) [Clang 21.0.0 (clang-2100.0.123.102)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/devishree/Desktop/DS1.py =====
Mean rating = 3.9
95% Confidence interval = (np.float64(3.188628400280501), np.float64(4.611371599719499))
>>>
```

Question 23: Hypothesis Testing for Treatment vs Placebo



Question 24: KNN Classifier for a New Patient

```
[22]
✓ 2s from sklearn.neighbors import KNeighborsClassifier

# Features: symptom1, symptom2
X = [
    [1, 1], [1, 2], [2, 1],
    [8, 8], [9, 8], [8, 9]
]

# 0 = No condition, 1 = Condition
y = [0, 0, 0, 1, 1, 1]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

patient = [[7, 8]]

prediction = model.predict(patient)

if prediction[0] == 1:
    print("Patient may have the condition")
else:
    print("Patient may not have the condition")

... Patient may have the condition
```

Question 25: Decision Tree for Iris Classification

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier(random_state=1)
model.fit(X, y)

sepal_length = float(input("Sepal length: "))
sepal_width = float(input("Sepal width: "))
petal_length = float(input("Petal length: "))
petal_width = float(input("Petal width: "))

new_flower = [
    sepal_length, sepal_width,
    petal_length, petal_width
]

prediction = model.predict(new_flower)

print("Predicted species:", iris.target_names[prediction[0]])

... Sepal length: 60.254
Sepal width: 30.567
Petal length: 40.789
Petal width: 20.465
Predicted species: virginica
```

Question 26: Linear Regression for Housing Price Prediction

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Area in square feet
X = np.array([[1000], [1200], [1500], [1800], [2000]])

# House prices
y = np.array([200000, 240000, 300000, 360000, 400000])

model = LinearRegression()
model.fit(X, y)

area = float(input("Enter house area: "))

prediction = model.predict([area])

print("Predicted house price =", prediction[0])

... Enter house area: 13.456
Predicted house price = 2691.19999999999425
```

Question 27: Logistic Regression for Customer Churn

```
[26] ✓ 29s
import numpy as np
from sklearn.linear_model import LogisticRegression

# Features: usage minutes, contract months
X = np.array([
    [100, 24],
    [120, 24],
    [80, 12],
    [300, 6],
    [280, 6],
    [350, 3]
])

# 0 = Not churned, 1 = Churned
y = np.array([0, 0, 0, 1, 1, 1])

# Create and train the model
model = LogisticRegression()
model.fit(X, y)

# Get new customer details
usage = float(input("Usage minutes: "))
contract = float(input("Contract duration: "))

# Predict churn
prediction = model.predict([[usage, contract]])

if prediction[0] == 1:
    print("Customer may churn")
else:
    print("Customer may not churn")

... Usage minutes: 12546
Contract duration: 54637
Customer may churn
```

Question 28: K-Means Customer Segmentation

```
[27] ✓ 0s
import numpy as np
from sklearn.cluster import KMeans

# Features: spending amount, number of purchases
X = np.array([
    [1000, 2],
    [1200, 3],
    [1100, 2],
    [5000, 10],
    [5200, 11],
    [4800, 9]
])

# Create and train the K-Means model
model = KMeans(n_clusters=2, random_state=1, n_init=10)
model.fit(X)

# New customer
new_customer = [[1500, 3]]

# Predict which cluster the customer belongs to
cluster = model.predict(new_customer)

print("Customer belongs to cluster:", cluster[0])

... Customer belongs to cluster: 0
```

Question 29: Accuracy, Precision, Recall and F1-Score

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# Actual values
y_test = [0, 1, 1, 0, 1, 0, 1, 1]

# Predicted values
y_pred = [0, 1, 0, 0, 1, 0, 1, 0]

print("Accuracy =", accuracy_score(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred))
print("Recall =", recall_score(y_test, y_pred))
print("F1-score =", f1_score(y_test, y_pred))

... Accuracy = 0.75
    Precision = 1.0
    Recall = 0.6
    F1-score = 0.75

```

Question 30: CART Regression for Used Car Price

```

import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Features: mileage, age
X = np.array([
    [20000, 2],
    [30000, 3],
    [40000, 4],
    [60000, 5],
    [80000, 7],
    [100000, 9]
])

# Car prices
price = np.array([18000, 17000, 15000, 13000, 10000, 7000])

# Create and train the model
model = DecisionTreeRegressor(max_depth=3, random_state=1)
model.fit(X, price)

# Get new car details
mileage = float(input("Mileage: "))
age = float(input("Age: "))

# Predict price
prediction = model.predict([[mileage, age]])

print("Predicted price =", prediction[0])

... Mileage: 1234
    Age: 30
    Predicted price = 7000.0

```

Question 31: Customer Segmentation using K-Means



Question 32: Bivariate Analysis and Linear Regression for House Price

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Size": [800, 1000, 1200, 1500, 1800, 2000],
    "Price": [160, 200, 240, 300, 360, 400]
})

X = data[["Size"]]
y = data[["Price"]]

model = LinearRegression()
model.fit(X, y)

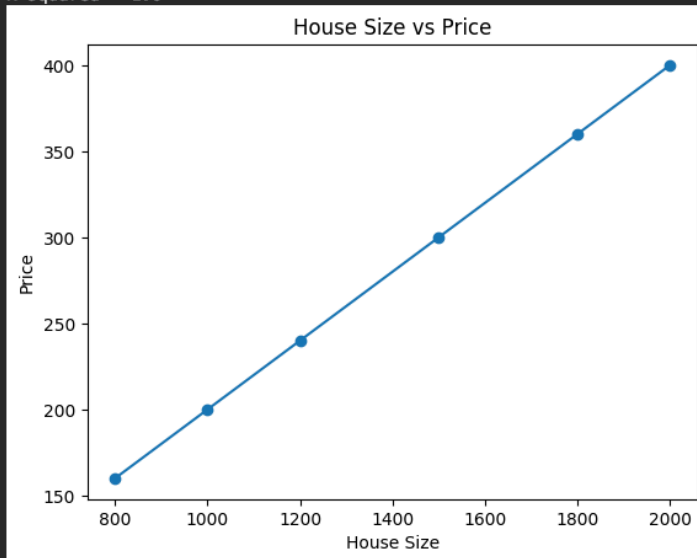
predicted = model.predict(X)

print("R-squared =", r2_score(y, predicted))

plt.scatter(data["Size"], data["Price"])
plt.plot(data["Size"], predicted)
plt.xlabel("House Size")
plt.ylabel("Price")
plt.title("House Size vs Price")
plt.show()

new_size = float(input("Enter house size: "))
print("Predicted price =", model.predict([[new_size]])[0])
```

R-squared = 1.0



Question 33: Multiple Linear Regression for Car Price

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Engine": [1.2, 1.5, 1.8, 2.0, 2.2, 2.5],
    "Horsepower": [80, 100, 120, 140, 160, 190],
    "Mileage": [20, 18, 16, 15, 13, 11],
    "Price": [8000, 10000, 13000, 16000, 19000, 23000]
})

X = data[["Engine", "Horsepower", "Mileage"]]
y = data["Price"]

model = LinearRegression()
model.fit(X, y)

predicted = model.predict(X)

print("R-squared =", r2_score(y, predicted))
print("Coefficients =", model.coef_)

new_car = pd.DataFrame({
    "Engine": [1.8],
    "Horsepower": [120],
    "Mileage": [16]
})

print("Predicted price =", model.predict(new_car)[0])

... R-squared = 0.9983049479376866
Coefficients = [-2692.30769231  180.76923077  115.38461538]
Predicted price = 12961.538461538466
```

Question 34: KNN Classification with Evaluation Metrics

```
[38] import numpy as np
      from sklearn.model_selection import train_test_split
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.metrics import (
          accuracy_score,
          precision_score,
          recall_score,
          f1_score
      )

      X = np.array([
          [20, 120],
          [22, 130],
          [25, 125],
          [27, 140],
          [40, 180],
          [42, 190],
          [45, 185],
          [48, 200]
      ])

      y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.25, random_state=1
      )

      model = KNeighborsClassifier(n_neighbors=3)
      model.fit(X_train, y_train)

      y_pred = model.predict(X_test)

      print("Accuracy =", accuracy_score(y_test, y_pred))
      print("Precision =", precision_score(y_test, y_pred))
      print("Recall =", recall_score(y_test, y_pred))
      print("F1-score =", f1_score(y_test, y_pred))

      new_patient = [[30, 150]]

      print("Prediction =", model.predict(new_patient))

... Accuracy = 1.0
   Precision = 1.0
   Recall = 1.0
   F1-score = 1.0
   Prediction = [0]
```

Question 35: K-Means Clustering for Retail Customers

```
[39] import pandas as pd
      from sklearn.cluster import KMeans

      data = pd.DataFrame({
          "Spending": [500, 700, 600, 4000, 4500, 4200, 1500, 1700],
          "Visits": [2, 3, 2, 10, 12, 11, 5, 6]
      })

      model = KMeans(n_clusters=3, random_state=1, n_init=10)

      data["Cluster"] = model.fit_predict(
          data[["Spending", "Visits"]]
      )

      print(data)

      print("Cluster centers:")
      print(model.cluster_centers_)

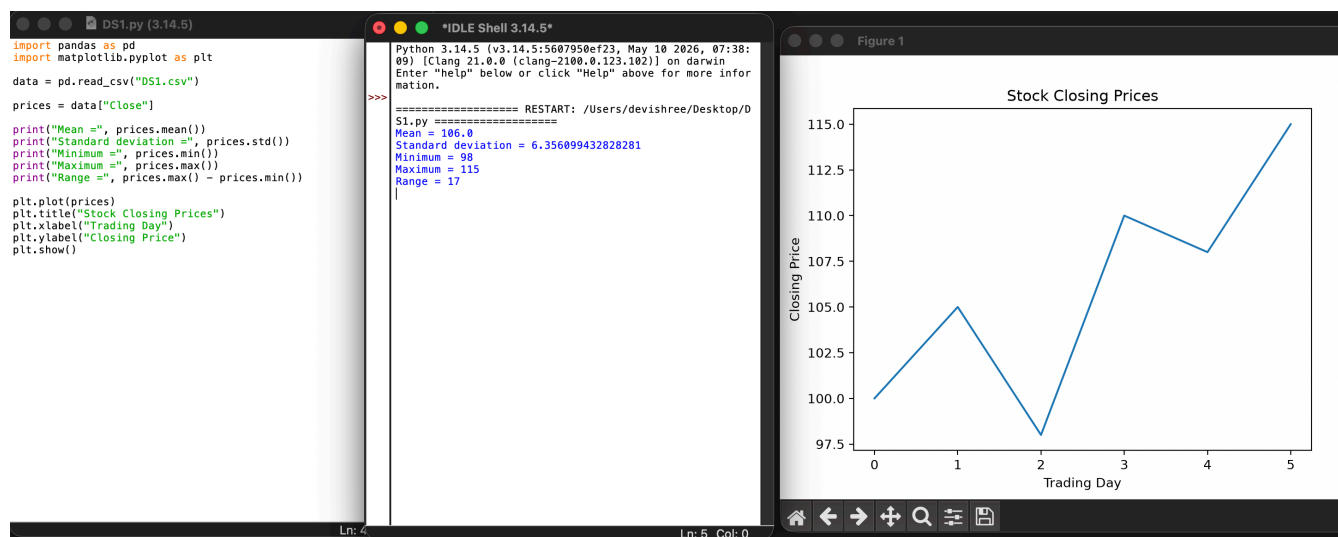
      ...
```

	Spending	Visits	Cluster
0	500	2	1
1	700	3	1
2	600	2	1
3	4000	10	0
4	4500	12	0
5	4200	11	0
6	1500	5	2
7	1700	6	2

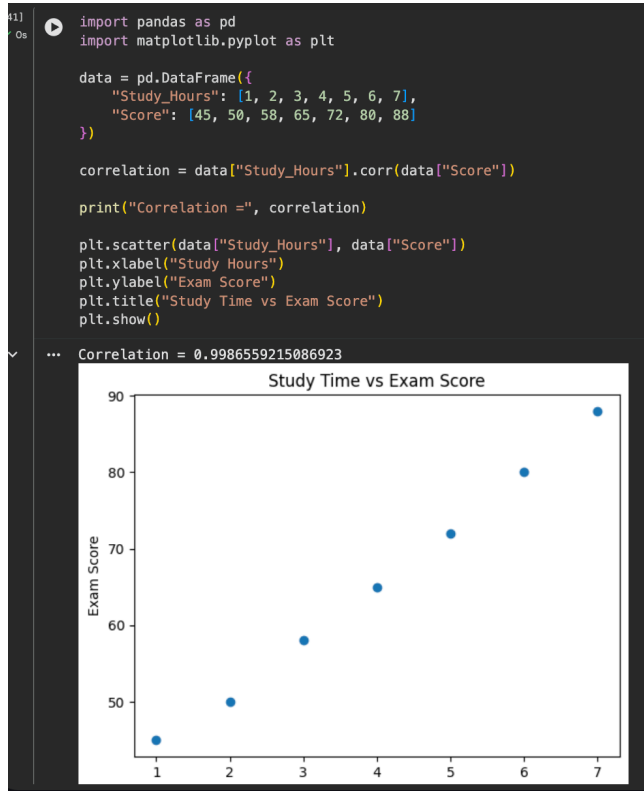
Cluster centers:

```
[[4.23333333e+03 1.10000000e+01]
 [6.00000000e+02 2.33333333e+00]
 [1.60000000e+03 5.50000000e+00]]
```

Question 36: Stock Price Variability from CSV



Question 37: Correlation between Study Time and Exam Score



Question 38: Temperature Analysis for Different Cities

```
[42] import numpy as np

temperatures = np.array([
    [25, 27, 30, 28, 26],
    [20, 21, 22, 21, 20],
    [30, 32, 35, 33, 31]
])

cities = ["Chennai", "Bengaluru", "Delhi"]

mean = np.mean(temperatures, axis=1)
std = np.std(temperatures, axis=1)
temperature_range = np.ptp(temperatures, axis=1)

for i in range(3):
    print(cities[i])
    print("Mean =", mean[i])
    print("Standard deviation =", std[i])
    print("Range =", temperature_range[i])

highest_range = np.argmax(temperature_range)
most_consistent = np.argmin(std)

print("Highest temperature range:", cities[highest_range])
print("Most consistent city:", cities[most_consistent])
```

... Chennai
Mean = 27.2
Standard deviation = 1.7204650534085253
Range = 5
Bengaluru
Mean = 20.8
Standard deviation = 0.7483314773547882
Range = 2
Delhi
Mean = 32.2
Standard deviation = 1.7204650534085253
Range = 5
Highest temperature range: Chennai
Most consistent city: Bengaluru

Question 39: K-Means Clustering and Visualization

```
[43]
✓ Os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "Spending": [500, 600, 550, 3000, 3200, 3100, 1500, 1600],
    "Items": [2, 3, 2, 10, 11, 9, 5, 6]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)

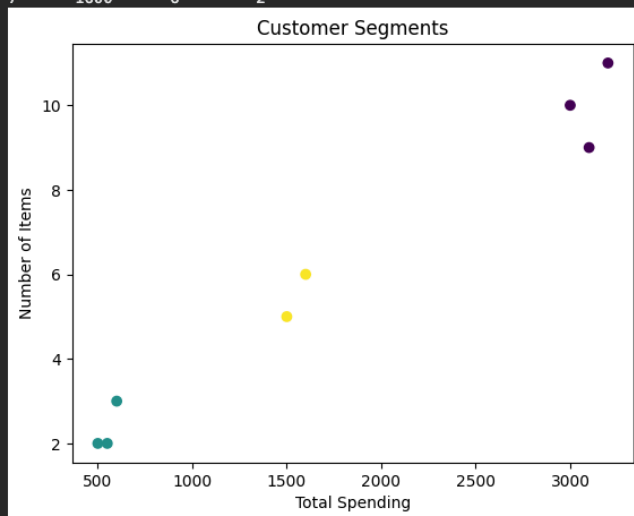
data["Cluster"] = model.fit_predict(
    data[["Spending", "Items"]]
)

print(data)

plt.scatter(
    data["Spending"],
    data["Items"],
    c=data["Cluster"]
)

plt.xlabel("Total Spending")
plt.ylabel("Number of Items")
plt.title("Customer Segments")
plt.show()
```

```
✓ ...
   Spending  Items  Cluster
0         500      2        1
1         600      3        1
2         550      2        1
3        3000     10        0
4        3200     11        0
5        3100      9        0
6        1500      5        2
7        1600      6        2
```



Question 40: Soccer Player Analysis using Pandas

